DIGT 2107
Lab 3 – Unit Testing

Due Date: February 6, 2026 – During Lab session

Your lab assignment is graded during the block week scheduled lab sessions.

1 Lab tasks

The goal of this lab is to practice the use of JUnit testing. You are given an archive file that represents a mini calculator. You are required to verify the functionality of the **Calculator**, **AdvancedCalculator**, classes. This practice will cover basic arithmetic operations, exponentiation, and logarithmic operations.

1.1 Calculator Class

Write a JUnit test case for each basic arithmetic operation method in the Calculator class (***add, subtract, multiply, divide***). Include test cases for edge cases such as zero values and large numbers. Ensure the division operation correctly throws an ArithmeticException when dividing by zero.

Expected test case tasks:

1) testAddition()

2) testSubtraction()

3) testMultiplication()

5) testDivision()

6) testDivisionByZero()

1.2 AdvancedCalculator Class

Extend the JUnit test cases for the Calculator class to cover the additional power cube and square sum methods in the AdvancedCalculator class.

Expected test case tasks:

7) testPower()

8) testCube()

9) testSqrtSumOfSquares ()

1.3 Test Coverage

For this task, you have to implement a comprehensive set of test cases for the given method in the enclosed example.
The given code is a working code that solves the problem of calculating reward points that are given to credit card customers, who use their card exceeding their credit limit. Suppose customers are given reward points instead of added charges. The points are given according to the table below. Assume that the expenditure is a whole value, i.e., there are no decimal numbers.

Expenditure over credit limit	Rewards
0 – 99 CAD	0 points
100 – 299 CAD	1 point for every dollar over the limit (100 -299 points)
300 – 499 CAD	2 points for every dollar over the limit (600 – 998 points)
Over 500 CAD	3 points for every dollar over the limit (over 1500 points)

Your job is to write a set of Junit test cases that comprehensively test the given code.
A comprehensive set of test cases is a set of tests that check all the paths in the code. If there is a conditional statement, both the true and false condition should be tested. If there are multiple conditions, all possible combination of the conditions being true or false should be tested. This means if the condition is "A AND B", 4 different test cases should be written, unless it is not possible. The 4 conditions are:
A=True, B=True – A=True, B=False - A=False, B=True – A=False, B=False

1.4 General Guidelines

Use assertions like assertEquals and assertThrows to validate the expected behavior. Ensure each test method is independent and does not rely on the state of other test methods.
Write meaningful test method names and include descriptive messages for failures. Make use of JUnit annotations such as @Test, @BeforeEach, or @AfterEach as needed.

2 Submission

Submit three files: **AdvancedCalculatorTest.java and CalculatorTest.java, TestingExampleTest.java** containing your JUnit tests to eclass!

**** Provide comments or explanations for any notable design decisions in your test cases.**

3 Grading Rubric

5% testAddition()
5% testSubtraction()
5% testMultiplication()
5% testDivision()
10% testDivisionByZero()

10% testPower()
10% testCube()
10% testSqrtSumOfSquares ()

40% TestCoverage (1.3)

Total 100%